

ACTIVITY – I

DATA DICTIONARY FOR AMAZON “YOU” PAGE

Data Dictionary:

A Data Dictionary in a Business Requirements Document (BRD) is a table that explains all the data fields used in the system.

Think of it as a **glossary for data**.

It describes:

- What each field means (example: Email = user's login email)
- What type of data it stores (text, number, date, etc.)
- How long the value can be (like 100 characters)
- Whether it is required or optional

Why is it needed in BRD?

Because it helps:

- Developers know what data to store
- Testers know what to validate
- Business teams understand the data structure



Amazon “You” Page – Data Dictionary

User Profile

Field Name	Description	Data Type	Length / Format	Mandatory
User id	Unique identifier for the user	UUID	36	Yes
full name	User's full legal name	VARCHAR	200	Yes
email	User's login email	VARCHAR	254	Yes
Phone number	User's primary phone number	VARCHAR	E 164 format	Yes
Date of birth	User's date of birth	DATE	YYYY-MM-DD	No
Preferred language	User's selected language	VARCHAR	10	No

ACTIVITY – II

WATERFALL, AGILE & V – MODEL

Waterfall Model:

The **Waterfall Model** is a step-by-step software development process where each phase must finish before the next one starts.

It flows downwards like a waterfall.

1. Requirement Gathering

- All requirements are collected at the beginning.
- Users tell exactly what the system should do.
- No development starts until requirements are fully documented.
- Clear and complete BRD/SRS is created.

2. System Design

- Designers create the overall architecture of the system.
- Technologies, databases, and modules are planned.
- Outputs: Design documents, diagrams, data models.
- Developers use these documents to write code.

3. Implementation (Coding)

- Developers write the code based on the design.
- Each module is developed separately.
- After coding, modules undergo basic testing.
- All modules are integrated together.

4. Testing

- Testers check if the system works as expected.
- Bugs are found and fixed.
- System is validated against requirements.
- Ensures quality before release.

5. Deployment

- System is delivered to the customer or end users.
- Installed in the production environment.
- Training may be provided to users.
- System becomes live.

6. Maintenance

- Fixes and updates after deployment.
- Enhancements based on new needs.
- Issues reported by users are resolved.
- Regular support and monitoring take place.

ACTIVITY – III

TEST EXECUTION PROCESS

TEST EXECUTION PROCESS – Demo Web Shop Website

1. Receive the Build

- QA receives the latest build/release of the Demo Web Shop website.
- Build is deployed to the QA/Test Environment.
- QA notes:
 - ✓ Environment URL
 - ✓ Login credentials
 - ✓ Build version
 - ✓ Browser/Device matrix

Example:

Demo Web Shop Build Version: v3.2.1

URL: <https://demowebshop.tricentis.com>

2. Test Case Design

The QA team prepares detailed test cases for all modules (Login, Search, Add to Cart, Wishlist, Checkout).

Below is the test case design format 5 designed test cases.

Test Case Design Table

TC No.	Module	Test Case Description	Preconditions	Test Steps	Test Data	Expected Result	Status
TC_001	Login	Verify login with valid credentials	User account exists	1. Open site 2. Click Login 3. Enter valid email & password 4. Click Login	Email: test@demo.com Pwd: Test@123	User successfully logs in and "My account" is displayed	Pass/Fail
TC_002	Login	Verify login with invalid password	None	1. Click Login 2. Enter valid email + wrong password 3. Click Login	Wrong password	Error message "Login was unsuccessful" appears	Pass/Fail
TC_003	Search	Verify search	None	1. Type "Laptop"	Keyword: Laptop	Laptop-related	Pass/Fail

ACTIVITY – IV

White Box Testing (Structural Testing)

White box testing means the tester **knows the internal code**, logic, and structure of the application.

- Tester needs programming knowledge.
- Internal logic, loops, and conditions are tested.
- Test cases are based on **code structure**.
- Used mostly by **developers**.
- Ensures code quality, flow, and paths are correct.

Advantages

- Helps find hidden coding errors.
- Optimization of code is possible.
- Covers all logical paths.
- Detects issues early in development.

Disadvantages

- Requires knowledge of code (not easy for all testers).
- Time-consuming.
- Not suitable for testing UI/UX.
- Complex for large applications.

Black Box Testing (Functional Testing)

Black box testing means the tester **does NOT know the internal code**, only tests the application from the **user's point of view**.

- Tester focuses on **input and output**, not code.
- No programming knowledge required.
- Functional behavior is tested.
- Real-world scenarios are checked.
- Most common testing done by QA teams.

Advantages

- Easy to perform, no coding skills needed.
- Tests application exactly like end users.
- Works well for large systems.
- Helps find missing requirements.

Disadvantages

- Cannot test code logic.
- Coverage may be low because internal code is hidden.
- Hard to design test cases without clear requirements.
- Not good for performance of internal components.

ACTIVITY – V

UAT (User Acceptance Testing)

- UAT is the **final testing phase** where the real users or clients test the application.
- They check whether the software **meets their business needs** and works the way they expect.
- It is done **after system testing** and before the product goes live.
- Users perform **real-world scenarios** to make sure everything works correctly.
- If users approve the application in UAT, the software is ready for **production release**.

Types of User Acceptance Testing (UAT)

Alpha User Acceptance Testing

- Performed **internally** by company employees before releasing to customers.
- Happens in a **controlled environment**.
- Goal is to catch major issues before customers use it.

Beta User Acceptance Testing

- Performed by **actual end users** in the real environment.
- Users try the product and give feedback.
- Helps identify issues that developers may have missed.

Black Box Testing (as part of UAT)

- Testers **do not see the internal code**, only check the system as a normal user.
- Focus is on **inputs and expected outputs**.
- Ensures the system behaves correctly from the user's point of view.
- Commonly used in UAT because business users focus on functionality.

Operational Acceptance Testing (OAT)

- Ensures the application works properly in the **production-like environment**.
- Checks backup, restore, performance, and monitoring.
- Often done by IT/operations team.
- Confirms the system can be **supported after release**.

Contract Acceptance Testing

- Done to verify that the software meets the **terms and conditions** written in the contract.
- If the product matches all contractual requirements → client accepts it.
- Common in government and vendor projects.

Regulation (Compliance) Acceptance Testing

- Ensures the software follows **legal rules, standards, and regulations**.
- Example: Privacy laws, safety standards, financial rules.
- Required for industries like healthcare, banking, insurance.